Exposé zur Bachelorthesis

Generative KI als Code-Analyse-Tool: Entwicklung und Evaluation eines LLM-Tools zur Erkennung von Qualitätsmängeln und Sicherheitslücken in Softwareanwendungen.

Felix Krauß

14. Oktober 2026

1 Zentrale Forschungsfrage

Inwieweit kann ein regelbasiertes, LLM-basiertes Analyse-Tool Qualitätsmängel und Sicherheitslücken in Softwareanwendungen zuverlässig erkennen, und wie schneidet es im Vergleich zu bestehenden Lösungen ab?

2 Zielsetzung der Arbeit

1. Überblick über den aktuellen Stand der Wissenschaft zur Verwendung generativer KI-Assistenten als Code-Analyse-Tools.

   Fragen, die dazu beantwortet werden sollen:

- Wie gut eignen sich laut dem derzeitigen Forschungsstand generative KI-Assistenten zum Erkennen von Qualitätsmängeln und Sicherheitslücken?
- Welche Standards und Kriterien eignen sich, um die Qualität von Software zu bewerten?
- Anhand welcher Kriterien und Klassifikationen werden Sicherheitslücken eingestuft?
- Welche Tools zur Evaluation von Sicherheit und Qualität gibt es bereits und was ist deren Funktionsweise?
- Was sagt der derzeitige Forschungsstand zur Effizienz von KI-Assistenten beim Analysieren von Softwareanwendungen und welche Modelle eignen sich für diese Aufgabe?

2. Welche Lösungen gibt es bereits, die generative KI-Assistenten als Analyse-Tool verwenden, und was sind laut Dokumentation deren Fähigkeiten und Limitationen? Basierend darauf und auf den Ergebnissen aus 1 festlegen, anhand welcher Kriterien und mithilfe welcher Werkzeuge ein eigenes Tool Qualität und Sicherheit bewertet.

3. Entwicklung eines eigenen Tools zum Analysieren von Quellcode. Das Tool erhält den Code einer Anwendung als Input und erstellt auf Basis vordefinierter Regeln mithilfe eines LLMs einen Bericht, der die Qualität und Sicherheit des Codes bewertet und Handlungsvorschläge

zur Verbesserung gibt.

Anforderungen an das Tool:

- Es gibt ein Regelwerk, das genau vorgibt, wie Qualität und Sicherheit des Codes gemessen werden. Das Tool orientiert seine Auswertung an diesen Regeln.
- Die Regeln sollen so definiert sein, dass diese auf den Code aller möglichen Anwendungen anwendbar sind, die auf den am weitesten verbreiteten Frameworks und Programmiersprachen basieren. Das Anpassen des Regelwerks an sich verändernde Anforderungen oder an spezifische Frameworks und Programmiersprachen soll nachträglich möglich sein.
- Das Tool soll lokal deploybar sein und unabhängig von dem verwendeten LLM funktionieren. Das bedeutet, das verwendete LLM soll austauschbar sein, und es soll möglich sein, sowohl ein Cloud-Modell eines Anbieters als auch ein selbst betriebenes LLM zu verwenden.

4. Test des eigenen Tools anhand mehrerer Beispiele, die bekannte Sicherheitslücken und Qualitätsmängel enthalten, unter Verwendung verschiedener LLMs. Anschließend werden die Ergebnisse ausgewertet und mit vorher definierten KPIs abgeglichen, z. B.: Mindestens 95 % der bekannten Qualitätsmängel werden entdeckt, alle als kritisch eingestuften Sicherheitslücken werden erkannt und maximal 10 % der Meldungen sind Fehlalarme. Derselbe Test wird mit anderen Analyse-Tools durchgeführt, deren Ergebnisse nach denselben Kriterien ausgewertet und mit den Ergebnissen des eigenen Tools verglichen werden.
5. (Optional) Einführung des Tools in die Infrastruktur der Hochschule, Dokumentation und eventuell Schulung zur Verwendung des Tools.

3 Aktueller Stand der Forschung

Die Verwendung von LLMs zur Analyse von Quellcode ist in den letzten Jahren stark angestiegen. Allein zur Erkennung von Sicherheitslücken wertet die systematische Literaturanalyse von Kaniewski et al. (2026, S. 43) 263 Studien aus. Ein Grund, aus dem vermehrt untersucht wird, inwieweit sich LLMs zum Erkennen von Sicherheitslücken eignen, ist, dass statische Analysewerkzeuge zwar weit verbreitet sind, aber häufig Fehlalarme produzieren, die von Hand geprüft werden müssen (Du et al., 2026, Abstract). Ein weiterer Grund ist, dass diese Tools zum Teil auf Spezifikationen angewiesen sind, die von Menschen definiert werden. Bei diesem Prozess passieren Fehler, und es können Spezifikationen übersehen werden (Z. Li et al., 2025, S. 1).

Als Maßstab für Qualität beschreibt der Standard ISO/IEC 25010 ein Modell mit neun Merkmalen, darunter Wartbarkeit und Sicherheit (ISO/IEC, 2023, S. 9). Die hier betrachteten Studien messen die Qualität der Ergebnisse, die LLMs produzieren, enger, etwa über das Bestehen von vordefinierten Unit-Tests (Cihan et al., 2025, Abschnitt I), ob von SonarQube gemeldete Issues korrekt gelöst werden (Patcas & Motogna, 2026, S. 1) oder über Trefferquoten gegenüber manuell erstellten Referenzen (Zeng et al., 2026, S. 1). Die meisten von Kaniewski et al. (2026, S. 43) betrachteten Studien klassifizieren sogar nur binär, ob eine Schwachstelle vorliegt oder nicht. In manchen Studien werden Sicherheitslücken über CWE als Katalog von Schwachstellentypen (Corporation, 2026), über CVE-Einträge zu bekannten Schwachstellen (Corporation, o. J.) und über den CVSS-Schweregrad

(Forum of Incident Response and Security Teams, Inc, o. J.) eingeordnet. Einige Arbeiten nutzen CWE auch als Ausgabeformat, das Modell nennt dann den Typ der Schwachstelle statt nur ja oder nein (Park et al., 2025, S. 1), was auch von Kaniewski et al. (2026, S. 42) empfohlen wird.

Wie gut LLMs Sicherheitslücken erkennen, wird unterschiedlich beurteilt. Taghavi Far & Feyzi (2025, S. 60) kommen zu dem Schluss, dass LLMs, unter geeigneten Bedingungen, statische Analysewerkzeuge übertreffen und zusätzlich Korrekturen vorschlagen können. Wichtig zu berücksichtigen ist, dass sie im Abstract ausdrücklich erwähnen, dass sie keinen direkten Vergleich vornehmen (Taghavi Far & Feyzi, 2025, S. 1). Zurückhaltender fällt das Bild bei Ding et al. (2025, S. 1736) aus: Dasselbe Modell schneidet dort je nach Datensatz völlig unterschiedlich ab, unter anderem weil ältere Datensätze falsche Labels und viele Duplikate enthalten; dadurch wird die Leistung überschätzt (Ding et al., 2025, S. 1729). Kaniewski et al. (2026, S. 38) nennen nebst vielen weiteren Limitationen genau diese Probleme bei den von Studien verwendeten Datensätzen. Außerdem erreichen, laut Ding et al. (2025, S. 1738) GPT-3.5 und GPT-4 beim Einstufen, ob es sich um verwundbaren oder bereits korrigierten Code handelt, nur eine Trefferquote, die auch bei zufälligem Auswählen plausibel wäre. Diese Aussage zu GPT-3.5 und GPT-4 lässt sich aber nicht ohne Weiteres auf aktuellere Modelle übertragen. Neuere Arbeiten zeigen, wie stark das Ergebnis vom Bewertungsrahmen abhängt: Erhalten Modelle wie DeepSeek-R1 im Strict Mode den Kontext des Codes, steigen Accuracy und Precision gegenüber der Bewertung isolierter Funktionen merklich, weshalb die Autoren die schlechten Werte früherer Studien auf fehlenden Kontext in der Evaluation zurückführen (Y. Li et al., 2025, Abschnitte 5.2–5.3). Dass ein richtiges Urteil allein wenig über die Qualität der Analyse aussagt, zeigen Lu et al. (2026, S. 8–9): In den dort ausgewerteten Ergebnissen beruhten bei zehn Modellen über ein Drittel der richtigen Urteile auf einer falschen Begründung; basierend darauf wird die Vermutung aufgestellt, dass bei früheren Studien die Ergebnisse dadurch verwässert wurden.

Für die Verwendung von LLMs zum Erkennen von Qualitätsmängeln und für Code-Reviews fällt das Urteil ähnlich zurückhaltend aus. Laut Cihan et al. (2025, Abstract) stuft GPT-4o die Korrektheit von Code in 68,50 % der Fälle richtig ein und Gemini 2.0 Flash in 63,89 % der Fälle, wenn die Problembeschreibung Teil des Kontextes war; aufgrund des Risikos fehlerhafter Ausgaben empfehlen die Autoren aber menschliche Reviews. Auf echten Pull Requests bricht die Leistung von LLMs ein, im SWR-Bench erreicht die beste Kombination, im Hauptvergleich, nur einen F1-Wert von 19,38 %, vor allem durch niedrige Precision-Werte (Zeng et al., 2026, S. 13). Bei Kumar et al. (2026, S. 1) erreicht das beste Modell auf realen Pull Requests einen F1-Wert von 0,066, auf synthetischen Beispielen einen Wert von 0,847, sie bestätigen damit den Eindruck, dass LLMs in echten Anwendungsfällen deutlich schlechter abschneiden. Außerdem können Code-Review-Agenten, wenn sie die Aufgabe bekommen, alle versteckten Probleme zu finden, laut Pereira et al. (2026, S. 1) im Verhältnis zu den tatsächlich relevanten Problemen viele unbegründete Befunde erzeugen. Patcas & Motogna (2026, S. 1) untersuchen die Eignung von LLMs für die Generierung von Fixes für die von SonarQube gefundenen Probleme. Im Mittel reduzieren die Fixes der LLMs die gefundenen Probleme um 36,02 %.

Als sinnvolle Anwendung in der Praxis wird die Kombination von LLMs mit klassischen statischen Analysen empfohlen. Kaniewski et al. (2026, S. 42) beschreiben Pipelines, in denen statische Werkzeuge Kandidaten liefern und das LLM Fehlalarme herausfiltert, oder umgekehrt die statische Analyse die Vorschläge des LLMs prüft, und auch Patcas & Motogna (2026, S. 42) sehen hybride

Systeme als möglichen nächsten Schritt. Konkrete Ansätze stützen das: IRIS nutzt eine Kombination aus statischen Analysen und LLMs, um Sicherheitslücken zu identifizieren, im Versuchsaufbau von Z. Li et al. (2025, S. 1) wird das Ergebnis von IRIS mit GPT-4 mit dem Ergebnis von CodeQL, einem weit verbreiteten Werkzeug für statische Code-Analyse, verglichen. IRIS hat dabei 55 Sicherheitslücken erkannt, während CodeQL nur 27 erkannte. In einer weiteren Industriestudie wird beobachtet, dass die getesteten hybriden Verfahren 94 bis 98 % der Fehlalarme beseitigen (Du et al., 2026, S. 716). LLift, ein Framework, das statische Analysen und LLMs kombiniert, mit dem Ziel, Bugs im Linuxkernel zu identifizieren, findet vier bislang unbekannte Fehler im Linuxkernel (H. Li et al., 2024, Abstract).

Für den Betrieb stellt sich zudem die Frage, ob ein lokales Modell infrage kommt. Im Hinblick auf die Anforderung der lokalen Deploybarkeit des Tools wäre ein lokales LLM besser, da so keine Daten an Cloud-Anbieter übertragen werden. Laut Park et al. (2025, S. 1) können angepasste lokale Modelle die Leistung von Cloud-Modellen übertreffen und ein besseres Kosten-Nutzen-Verhältnis erreichen. Laut Kaniewski et al. (2026, S. 42) gilt das Feintuning kleiner Modelle für den Betrieb im eigenen Haus als praktikabel.

Es gibt Ansätze, die zeigen, dass größere Modelle nicht zwingend bessere Code-Reviews erzeugen: Im Code-Review schlägt das kleinere Claude Haiku 4.5 das größere Sonnet 4.6 bei 3,2-fach geringeren Kosten (Kumar et al., 2026, S. 1), und mehrere Durchläufe kleinerer Modelle erreichen die Leistung eines größeren (Zeng et al., 2026, S. 20). IRIS liefert einen Gegenbefund, denn die Kontextanalyse hilft dort nur bei den größeren Modellen (Z. Li et al., 2025, S. 9).

3.1 Schlüsse aus der Forschung und mögliche Auswirkungen auf das zu entwicklende Tool

Die betrachteten Studien legen nahe, dass LLMs als Analysewerkzeug vielversprechend sind, die Kombination mit statischer Analyse und menschlicher Kontrolle die Zuverlässigkeit verbessern kann und Tests mit synthetischen Daten dazu führen können, dass die Leistung der LLMs überschätzt wird. Außerdem kann man aus den Studien die Hypothese ableiten, dass lokale LLMs mit der richtigen Feinabstimmung genauso gut oder sogar besser geeignet sind wie Cloud-Modelle. Für das zu entwickelnde Tool bedeutet das, dass ein lokales feinabgestimmtes LLM in Kombination mit statischen Analyse-Tools einen realistischen Lösungsansatz darstellt. Für die Tests des Tools sollten, wenn möglich, reale Daten herangezogen werden, da synthetische Daten die Leistung des Tools überschätzen könnten. Da das Tool nur einen Bericht mit Handlungsempfehlung generieren soll und nichts direkt geändert wird, bleibt eine menschliche Kontrolle sowieso unerlässlich.

4 Vorläufige Gliederung

1. Einleitung
   1.1 Motivation und Problemstellung
   1.2 Forschungsfrage und Zielsetzung
   1.3 Abgrenzung und Aufbau der Arbeit

2. Grundlagen
   2.1 Standards und Kriterien für Softwarequalität
   2.2 Sicherheitslücken und Klassifikationen
   2.3 Statische Code-Analyse
   2.4 Betrieb und Funktion von Large Language Models
3. Stand der Forschung
   3.1 Verwendung von LLMs zur Erkennung von Sicherheitslücken und Qualitätsmägneln
   3.2 Hypbride Ansätze (statische Analys + LLM)
   3.3 Eignung lokaler Modelle
   3.4 Limitationen bisheriger Sutdien
   3.5 Fazit und Konsquenzen für das eigene Tool
4. Prüfrahmen des eigenen Tools
   4.1 Überblick über ähnliche vorhandene Lösungen
   4.2 Kriterien für Qualität und Sicherheit
   4.3 Anforderungen an das Regelwerk.
5. Entwicklung des Tools
   5.1 Architektur
   5.2 Regelwerk und Erweiterbarkeit des Regelwerks
   5.3 Austauschbarkeit des LLMs
   5.4 Integration in das lokale System
   5.5 Umsetzung
6. Evaluation
   6.1 Evaluationsdesign
   6.2 Vergleichswerkzeuge und Vorgehen
   6.3 Auswertung der Ergebnisse
   6.4 Vergelich mit bestehenden Tools
7. Fazit und Ausblick
   7.1 Beantwortung der Forschungsfrage
   7.2 Ausblick

Vorläufiges Literaturverzeichnis

Cihan, U., İçöz, A., Haratian, V., & Tüzün, E. (2025). *Evaluating Large Language Models for Code Review* (arXiv:2505.20206). arXiv. https://doi.org/10.48550/arXiv.2505.20206

Corporation, T. M. (o. J.). *Common Vulnerabilities and Exposures (CVE)*. https://www.cve.org/About/Metrics.

Corporation, T. M. (2026). *CWE - Common Weakness Enumeration*. https://cwe.mitre.org/.

Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Alomair, B., Wagner, D., Ray, B., & Chen, Y. (2025). Vulnerability Detection with Code Language Models: How Far Are We? *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 1729–1741. https://doi.org/10.1109/ICSE55347.2025.00038

Du, X., Feng, J., Zou, Y., Xu, W., Ma, J., Zhang, W., Liu, S., Peng, X., & Lou, Y. (2026). Reducing False Positives in Static Bug Detection with LLMs: An Empirical Study in Industry. *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*, 716–727. https://doi.org/10.1145/3786583.3786910

Forum of Incident Response and Security Teams, Inc. (o. J.). *Common Vulnerability Scoring System SIG*. https://www.first.org/cvss/.

ISO/IEC. (2023). *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model* (ISO/IEC 25010:2023).

Kaniewski, S., Schmidt, F., Enzweiler, M., Menth, M., & Heer, T. (2026). A Systematic Literature Review on Detecting Software Vulnerabilities with Large Language Models. *ACM Transactions on Software Engineering and Methodology*, 3815425. https://doi.org/10.1145/3815425

Kumar, S. P., Bararia, S., & Raj, K. (2026). *Bigger Isn't Always Better: A Comparative Evaluation of LLMs for Automated Code Review* (arXiv:2606.15689). arXiv. https://doi.org/10.48550/arXiv.2606.15689

Li, H., Hao, Y., Zhai, Y., & Qian, Z. (2024). Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach. *Proceedings of the ACM on Programming Languages*, *8*(OOPSLA1), 474–499. https://doi.org/10.1145/3649828

Li, Y., Li, X., Wu, H., Xu, M., Zhang, Y., Cheng, X., Xu, F., & Zhong, S. (2025). *Everything You Wanted to Know About LLM-Based Vulnerability Detection But Were Afraid to Ask* (arXiv:2504.13474). arXiv. https://doi.org/10.48550/arXiv.2504.13474

Li, Z., Dutta, S., & Naik, M. (2025). IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In Y. Yue, A. Garg, N. Peng, F. Sha, & R. Yu (Hrsg.), *International Conference on Learning Representations* (Bd. 2025, S. 35735–35758).

Lu, L., Zhao, Y., Rao, H., Zhang, K., & Wang, H. (2026). *Evaluating and Enhancing the Vulnerability Reasoning Capabilities of Large Language Models* (arXiv:2602.06687). arXiv. https://doi.org/10.48550/arXiv.2602.06687

Park, S., Ko, G., & Cho, H. (2025). *On the Effectiveness of Instruction-Tuning Local LLMs for Identifying Software Vulnerabilities* (arXiv:2512.20062). arXiv. https://doi.org/10.48550/arXiv.2512.20062

Patcas, R., & Motogna, S. (2026). An Evaluation Study of Large Language Models for Addressing Code Quality Issues. *Empirical Software Engineering*, *31*(5), 118. https://doi.org/10.1007/s10664-026-10858-8

Pereira, K., Sinha, N., Ghosh, R., & Dutta, D. (2026). *CR-Bench: Evaluating the Real-World Utility of AI Code Review Agents* (arXiv:2603.11078). arXiv. https://doi.org/10.48550/arXiv.2603.11078

Taghavi Far, S. M., & Feyzi, F. (2025). Large Language Models for Software Vulnerability Detection: A Guide for Researchers on Models, Methods, Techniques, Datasets, and Metrics. *International Journal of Information Security*, *24*(2), 78. https://doi.org/10.1007/s10207-025-00992-7

Zeng, Z., Shi, R., Han, K., Li, Y., Sun, K., Wang, Y., Yu, Z., Xie, R., Ye, W., & Zhang, S. (2026). SWR-Bench: Assessing LLM Performance in Real-World Code Review Comment Generation. *Proceedings of the ACM on Software Engineering*, *3*(FSE), 3093–3115. https://doi.org/10.1145/3808144